# REAL TIME HYBRID INTRUSION DETECTION SYSTEM USING MACHINE LEARNING

# ABSTRACT

In today's rapidly evolving digital landscape, cybersecurity threats have become increasingly sophisticated, targeting both network infrastructure and individual host systems simultaneously. Traditional Intrusion Detection Systems often focus on either network-level or host-level threats, leaving critical gaps in security coverage. This project proposes and implements a Hybrid Intrusion Detection System that integrates multiple layers of threat detection into a unified, real-time monitoring platform.

The proposed system combines a Network-based Intrusion Detection System (NIDS) powered by a Random Forest AI classifier trained on the UNSW-NB15 benchmark dataset a real-world cybersecurity dataset from the University of New South Wales, Australia, containing over 175,000 records of both normal and attack traffic across 9 attack categories. The AI model is then deployed to classify live network traffic captured in real time using the Scapy library. This approach mirrors how professional IDS tools like Snort and Suricata operate training on known attack data, then detecting live traffic.

The system simultaneously runs a Host-based Intrusion Detection System (HIDS) that monitors file integrity, suspicious running processes, and failed login attempts on the host machine. Additionally, the system incorporates Web Threat Detection using the VirusTotal API (70+ antivirus engines) and Google Safe Browsing API to identify malicious URLs, phishing websites, and malware-infected files.

The entire system is presented through an interactive Streamlit dashboard with 7 pages providing live packet logs, threat alerts, model performance metrics including accuracy, precision, recall, and F1 score, and email notification support. Experimental results using UNSW-NB15 demonstrate that the Random Forest model achieves high and realistic accuracy (95–99%) in classifying both normal and attack traffic.

# TABLE OF CONTENT

# 1. INTRODUCTION

## 1.1 PROJECT INTRODUCTION

In the modern era of computing and deeply interconnected digital systems, cybersecurity has emerged as one of the most pressing and critical challenges faced by individuals, organizations, enterprises, and governments across the globe. Every day, thousands of cyber-attacks target network infrastructure, personal computers, corporate servers, and web applications with the intent to steal sensitive data, disrupt services, or compromise entire systems. The increasing frequency, scale, and sophistication of these attacks have made it absolutely essential to develop and deploy intelligent, multi-layered security solutions that are capable of detecting threats at multiple levels simultaneously and responding in real time.

The Hybrid Intrusion Detection System (Hybrid IDS) is an innovative and advanced security platform developed to address this critical need. The system uniquely integrates three distinct and complementary layers of threat detection into a single, unified, real-time monitoring dashboard: Network-based Intrusion Detection (NIDS), Host-based Intrusion Detection (HIDS), and Web Threat Detection. By combining all three approaches into one cohesive platform, this project delivers comprehensive security coverage that no traditional single-layer IDS solution can achieve. The integration of these three layers means that threats attacking the system at the network level, at the host level, and through web channels can all be detected, logged, and alerted upon simultaneously without requiring the security analyst to switch between multiple disconnected tools.

**Network-Based Intrusion Detection System (NIDS):**

The NIDS component is powered by a Random Forest artificial intelligence classifier trained on the UNSW-NB15 benchmark dataset a real-world, professionally curated cybersecurity research dataset created at the University of New South Wales, Australia, containing 175,341 records of both normal and attack network traffic spanning 9 distinct attack categories. The trained model is deployed in the Live Monitor dashboard page to classify live network packets captured in real time using the Scapy packet capture library.

For each captured packet, nine key numerical features are extracted and passed to the Random Forest model, which instantly produces a NORMAL or ATTACK classification along with a confidence percentage. This methodology mirrors exactly how professional, enterprise-grade IDS tools such as Snort and Suricata operate in real-world deployments by training offline on known attack data and deploying the trained intelligence online for live traffic analysis.

**Host-Based Intrusion Detection System (HIDS):**

The HIDS component operates as three simultaneous background monitoring threads that run continuously alongside the dashboard without interrupting its operation. The watchdog library monitors the file system in real time, raising immediate HIGH severity alerts whenever suspicious files such as executables (.exe), scripts (.bat, .ps1), or dynamic link libraries (.dll) are created or modified on the Desktop or in the Documents directory. The Psutil library scans all running processes every ten seconds, cross-referencing each process name against a list of known hacking and reconnaissance tools including Mimikatz, Netcat, Nmap, Metasploit, and SQLMap. The Windows Security Event Log is queried every thirty seconds for Event ID 4625 (failed login attempt), and multiple failures within a short time window trigger a brute-force attack alert with HIGH severity. All HIDS alerts are categorized as HIGH, MEDIUM, or LOW severity and displayed on the dedicated HIDS Monitor dashboard page with full timestamp and contextual detail.

**Web Threat Detection Layer:**

The Web Threat Detection layer provides on-demand URL and file scanning by simultaneously querying two industry-leading threat intelligence services. The VirusTotal API v3 checks any submitted URL against more than 70 antivirus engines and security databases, returning the number of engines that flagged the URL as malicious, suspicious, or clean. The Google Safe Browsing API v4 cross-references the URL against Google's own database of billions of known phishing, malware, and social engineering websites in the same database that powers the built-in safety features of Google Chrome and Mozilla Firefox. For file scanning, the SHA-256 cryptographic hash of the file is computed locally on the user's machine and only the hash string is transmitted to VirusTotal, ensuring that the actual file contents never leave the system and complete privacy is maintained.

The entire system is presented through a seven-page interactive Streamlit dashboard that provides intuitive sidebar navigation, color-coded threat alerts, real-time packet monitoring logs, AI model performance metrics, email-based notification support, and a Manual Packet Predict page for demonstration during academic viva examinations. The system is implemented entirely using freely available, open-source Python libraries, requiring zero licensing investment and no specialized hardware, making it a practically deployable solution for academic institutions, small organizations, and individual security researchers alike.

## 1.2 EXISTING SYSTEM

### Network-Based IDS Tools (NIDS):

The most widely deployed NIDS tools are Snort and Suricata. Both are rule-based systems that inspect network traffic against predefined attack signatures. While effective at detecting known attacks, they operate exclusively at the network layer with no visibility into host-level events such as suspicious files or brute-force login attempts. Their raw log outputs also require significant technical expertise to interpret.

### Host-Based IDS Tools (HIDS):

OSSEC is a widely used open-source HIDS that monitors log files, checks file integrity, and detects rootkits. Tripwire is a commercial tool that alerts when cryptographic hashes of critical system files change. While both tools excel at host-level monitoring, they are blind to network-level attacks and web-based threats,and require significant manual configuration with no graphical interface for non-specialist users.

### Web Threat Detection Tools:

Existing web threat detection relies on separate, standalone tools. Browser-based protections like Google Safe Browsing operate silently with no centralized alerting or integration with network monitoring. Third-party scanners like VirusTotal require manual URL submission with no automation.

Enterprise platforms that unify all three monitoring types carry annual licensing costs far beyond the reach of academic institutions and small organizations.

**Limitations of AI Integration in Existing Systems:**

Most open-source IDS tools rely on rule-based detection, requiring security experts to manually write new signatures for each discovered attack. This reactive approach fails against zero-day and novel attacks. Machine learning-based alternatives are either expensive commercial products or academic prototypes without practical interfaces, leaving a significant gap in freely available, AI-powered multi-layer IDS tools.

## 1.3 DRAWBACKS OF EXISTING SYSTEM

**No Unified Dashboard:** Existing NIDS and HIDS tools are completely separate systems with no unified interface. Analysts must switch between multiple tools and manually correlate alerts a time-consuming, error-prone process that can cause threats to go unnoticed.

**High Cost and Complexity:** Enterprise IDS solutions such as IBM QRadar, Splunk SIEM, and Palo Alto Cortex XDR carry annual licensing costs ranging from tens of thousands to hundreds of thousands of rupees, requiring dedicated hardware and certified professionals completely out of reach for academic institutions and small organizations.

**No Web Threat Integration:** Tools such as Snort, Suricata, OSSEC, and Tripwire have no integration with web threat intelligence. Phishing URLs, drive-by malware, and social engineering attacks are invisible to these systems and must be handled by separate, unconnected tools.

**Rule-Based Detection Cannot Adapt:** Rule-based IDS tools can only detect attacks with an existing signature. Zero-day and novel attack variations pass through undetected, and keeping signature databases current requires continuous manual effort by security experts.

**No Real-Time Notification System:** Most open-source IDS solutions log alerts to text files or system logs without proactively notifying the administrator. The analyst must actively check logs, meaning attacks occurring outside working hours may go undetected for extended periods.

**Poor and Technical User Interfaces:** Tools-like Snort, Suricata, and OSSEC output raw log files and command-line alerts requiring deep technical expertise. Without additional commercial visualization tools, these outputs are inaccessible to non-specialist users and students.

**No Demonstration or Testing Facility:** Existing IDS tools have no facility for manually injecting test packets or simulating attacks without generating actual malicious traffic making academic demonstrations and viva presentations ethically and practically difficult.

**Specialized Hardware Requirements:** Enterprise IDS deployments typically require dedicated hardware appliances, network taps, and span ports on managed switches. These prerequisites involve significant capital expenditure and are unavailable in most academic or small business environment.

## 1.4 PROPOSED SYSTEM

The proposed Hybrid Intrusion Detection System addresses every limitation of existing solutions by providing a unified, AI-powered, multi-layer security platform built entirely from open-source Python components, deployable on any standard personal computer without specialized hardware.

**Automated Training Data Preparation:**

The capture.py module automatically reads the UNSW-NB15 dataset, maps 49 columns to 9 standardized packet feature fields, converts labels to binary normal/attack format, samples 50,000 balanced records, and outputs a clean live_traffic.csv training file eliminating complex data preprocessing that typically requires machine learning expertise.

**AI Model Training and Selection:**

The train_model.py module splits the data into 80/20 training and testing sets using stratified sampling and trains two classifiers: a Random Forest Classifier (RFC) and an Extra Trees Classifier (ETC), each with 100 decision trees. The better-performing model is automatically saved as ids_model.pkl along with metrics and a confusion matrix heatmap for performance evaluation.

**Live Network Monitoring (NIDS Layer):**

The Live Monitor page uses Scapy to capture real network packets from the active interface. For each IP packet, nine features are extracted from the header and passed to ids_model.pkl, which classifies the packet as NORMAL or ATTACK with a confidence percentage in under 5 milliseconds. Results appear in a color-coded log (green/red) with timestamp, IP details, protocol, and confidence, alongside an auto-updating traffic distribution pie chart.

**Host Intrusion Detection (HIDS Layer):**

Clicking START HIDS launches three background daemon threads simultaneously. The File Integrity Monitor (watchdog library) raises HIGH severity alerts for suspicious file extensions (.exe, .bat, .ps1, .dll, .vbs) on the Desktop and Documents directories. The Process Monitor (psutil) scans running processes every 10 seconds against known attacker tools such as Mimikatz, Nmap, and Metasploit. The Login Monitor queries Windows Event Log every 30 seconds for Event ID 4625 (failed logins), triggering brute-force alerts when a threshold is exceeded without blocking the dashboard.

**Web Threat Detection Layer:**

The Web Threat Scanner page provides on-demand URL and file analysis. URL submissions trigger simultaneous calls to the VirusTotal API v3 (70+ antivirus engines) and Google Safe Browsing API v4. For file scanning, only the SHA-256 hash is sent to VirusTotal the actual file never leaves the device. Results appear within 2 - 8 seconds as a green, CLEAN or red THREAT DETECTED badge.

**Real-Time Email Alert System:**

The Alert Settings page allows the user to configure Gmail SMTP notifications using a sender address, App Password, and recipient address. The system automatically sends HTML-formatted email alerts via STARTTLS encryption whenever any threat is detected across the three monitoring layers.

## 1.5 ADVANTAGES OF PROPOSED SYSTEM

**Complete 360-Degree Security Coverage:**

The system simultaneously monitors network traffic (NIDS), host system events (HIDS), and web-based threats (Web Threat Detection) in a single unified dashboard, providing security coverage across all three attack surfaces that no existing single-tool solution offers.

**AI-Powered Adaptive Detection:**

The Random Forest classifier trained on 175,341 UNSW-NB15 records learns statistical patterns across 9 attack categories, enabling behavior-based detection far more adaptable to novel attack variations than any signature-based tool.

**High and Realistic Detection Accuracy:**

The model achieves 95–99% accuracy, precision, recall, and F1 score on the UNSW-NB15 test set metrics that reflect genuine detection capability on real-world attack data rather than artificially inflated accuracy from training on clean-only traffic as seen in naive implementations.

**Zero Cost Entirely Free and Open Source:**

Every tool, library, dataset, and API used in this project is freely available with no licensing cost. Python, Streamlit, Scapy, scikit-learn, Pandas, NumPy, watchdog, and psutil are all open-source. The UNSW-NB15 dataset and the free tiers of VirusTotal and Google Safe Browsing APIs are more than sufficient for academic use.

**Intuitive 7-Page Streamlit Dashboard:**

The Streamlit dashboard provides an accessible, browser-based graphical interface with sidebar navigation, color-coded threat badges, live Plotly charts, interactive metric cards, and plain-English alert descriptions making the system usable by both security professionals and non-specialist users without requiring any technical command-line expertise beyond launching the application.

**Manual Packet Predict for Academic Demonstration:**

The Manual Predict page allows users to enter packet feature values directly and receive an instant AI classification with confidence percentage enabling reliable demonstration during through examinations without generating actual attack traffic.

**Transparent Model Performance Evaluation:**

The Model Performance page displays accuracy, precision, recall, and F1 score, a classifier comparison bar chart, a radar chart, a confusion matrix heatmap, and a per-class classification report providing full academic visibility into the AI model's quality.

**Automated Real-Time Email Notification:**

The system automatically dispatches HTML formatted email alerts through Gmail SMTP the moment any threat is detected across any monitoring layer, ensuring the security analyst is notified immediately even when not actively monitoring the dashboard a critical capability that most open-source IDS tools completely lack.

**Deployable on Standard Consumer Hardware:**

The system runs on any standard Windows 10/11 laptop with a minimum of 4GB RAM and 10GB free disk space. No dedicated server, network tap, or specialized hardware is required, making it immediately deployable in any academic or office environment.

# 2. SYSTEM SPECIFICATION

## 2.1 HARDWARE SPECIFICATION

- Minimum 4 GB RAM required for running Streamlit dashboard, Scapy packet capture, and Random Forest model simultaneously.
- 16 GB RAM is ideal for running additional security tools alongside the Hybrid IDS dashboard without memory contention.
- 8 GB RAM recommended for smooth performance during live capture sessions with 500 packets and concurrent HIDS background monitoring threads.

**Storage:**

- Minimum 10 GB free disk space required for Python virtual environment, installed libraries, UNSW-NB15 training dataset, and generated PKL model files.
- SSD (Solid State Drive) recommended HDD for faster model loading, CSV reading, and dashboard startup times.
- Additional 5 GB recommended for storing captured packet logs and HIDS alert history over extended monitoring sessions.

**Network Interface Card (NIC):**

A functional Ethernet or Wi-Fi network adapter is mandatory for live packet capture using the Scapy library. The NIC must support promiscuous mode to allow the system to capture all packets passing through the network interface, not just packets addressed to the host machine. Both wired (Ethernet) and wireless (802.11 Wi-Fi) adapters are supported.

**Operating System:**

- Windows 10 (64-bit) or Windows 11 (64-bit) is a primary platform on which the project was developed and tested.
- Windows 11 is recommended as it provides improved security event logging through Windows Security Event Log, which is used by the HIDS Login Monitor module (Event ID 4625 for failed login detection).
- Linux (Ubuntu 20.04 or above) can also be used with minor path adjustments in hids_monitor.py for file system monitoring directories.

**Internet Connection:**

- A stable internet connection with minimum 2 Mbps speed is required for the Web Threat Scanner module to communicate with the VirusTotal API and Google Safe Browsing API.
- Internet access is also required during the model training phase if Python libraries need to be downloaded or updated.
- The Live Network Monitor and HIDS Monitor modules function without internet access, as they operate on local traffic and local system events.

**System Privileges:**

Administrator or root-level privileges are mandatory for Scapy to access the raw network socket interface on the host machine. Without Administrator rights, the Live Network Monitor will fail to capture packets. On Windows, PowerShell must be launched using 'Run as Administrator' before executing the application.

## 2.2 SOFTWARE SPECIFICATION

**Programming Language Python 3.12:** The core language for the entire project, selected for its extensive cybersecurity, machine learning, and networking library ecosystem. All required libraries Scapy, scikit-learn, Streamlit, watchdog, and psutil are fully compatible with Python 3.12.

**Web Dashboard - Streamlit 1.35.0:**

- Powers all 7 dashboard pages: Home, Live Monitor, Manual Predict, Model Performance, Web Threat Scanner, HIDS Monitor, and Alert Settings.
- Session state (st.session_state) maintains packet logs and settings across page navigations without a database.

**Packet Capture - Scapy:**

- Captures live packets from the network interface using the sniff () function in the NIDS layer.
- Extracts 9 packet fields: protocol, src_port, dst_port, pkt_len, ttl, ihl, tos, frag_offset, tcp_flags.

**Machine Learning - Scikit-learn:**

- Provides Random Forest Classifier and Extra Trees Classifier for AI model training and evaluation.
- Evaluation functions: accuracy_score(), precision_score(), recall_score(), f1_score(), confusion_matrix().

**Data Processing - Pandas and NumPy:**

- Pandas read the UNSW-NB15 CSV (175,341 rows), maps columns, samples 50,000 records, saves live_traffic.csv.
- NumPy constructs feature arrays for the Manual Predict page in the format expected by scikit-learn.

**Model Serialization - Joblib:**

- Saves trained model and metadata (ids_model.pkl, feature_cols.pkl, label_encoder.pkl, model_metrics.pkl).
- Reloads all objects at dashboard startup using joblib.load().

**Host Monitoring - Watchdog and Psutil:**

- Watchdog monitors Desktop and Documents directories for suspicious file events (.exe, .bat, .ps1, .dll).

- Psutil scans all running processes every 10 seconds against a list of known hacking tools.

**Web Threat APIs - Requests Library:**

- VirusTotal API v3: checks URLs and file hashes against 70+ antivirus engines (500 free requests/day).
- Google Safe Browsing API v4: checks URLs against Google's phishing and malware database (10,000 free requests/day).

**Email Alerts - Smtplib:**

- Sends HTML-formatted email alerts via Gmail SMTP (port 587, STARTTLS) when any threat is detected.
- Uses a 16-character Gmail App Password for secure programmatic authentication.

**Supporting Libraries:**

- Plotly - interactive pie chart (Live Monitor), bar chart and radar chart (Model Performance page).
- Hashlib (built-in) - computes SHA-256 hash of files locally; only hash is sent to VirusTotal for privacy.
- Threading (built-in) - runs HIDS file, process, and login monitors as simultaneous background daemon threads.

**Dataset - UNSW-NB15:**

- Created by the Australian Centre for Cyber Security (ACCS), UNSW Canberra, 2015.
- 175,341 records covering 9 attack types: Fuzzers, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms, Analysis.

**Development Tools:**

- Visual Studio Code - primary IDE with Python and Pylance extensions for development and debugging
- PowerShell (Administrator mode) - used to activate the virtual environment and launch the dashboard.

# 3. FEASIBILITY STUDY

## 3.1 TECHNICAL FEASIBILITY

The Hybrid IDS project is technically feasible as all required technologies are well-established, freely available, and have been successfully tested. Python 3.12 provides a mature ecosystem covering all required domains: networking (Scapy), machine learning (scikit-learn), data processing (pandas, numpy), web applications (Streamlit), and system monitoring (watchdog, psutil).

The UNSW-NB15 dataset is publicly available for academic research and provides the labeled data needed for proper AI model training with realistic examples of both normal and attack traffic. The Random Forest classifier in scikit-learn successfully trains on 50,000 sampled records within seconds on standard hardware, requiring no GPU or specialized computation.

The VirusTotal API and Google Safe Browsing API both offer free tiers sufficient for academic demonstration purposes. The project was implemented and tested on Windows 11 with Python 3.12, confirming full technical feasibility on standard consumer hardware. The Streamlit dashboard renders correctly in all modern web browsers without any special plugins.

## 3.2 BEHAVIOUR FEASIBILITY

Behavioral feasibility evaluates whether the Hybrid IDS will be accepted and effectively used by its target audience. The system is designed with simplicity as its primary goal the Streamlit dashboard requires no prior cybersecurity training to operate, and all results are presented with plain English color-coded alerts.

**User Acceptance:**

All five user categories (Security Analyst, System Administrator, Network Administrator, Management, and Student/Researcher) are expected to adopt the system with high acceptance, as it enhances rather than replaces existing security workflows. The dashboard's single-click operation and intuitive sidebar navigation minimize the learning curve.

**Organizational Impact:**

- Reduces manual monitoring effort through automated threat detection and email alerts
- Improves security posture by monitoring network, host, and web threats simultaneously
- Zero licensing cost - entirely open-source tools reduce operational expenses
- Serves as a learning platform for staff to understand attack patterns

**Change Management:**

A 30-minute walkthrough is sufficient to train new users across all 7 dashboard pages. Since the system assists human judgment rather than replacing it, user resistance is expected to be minimal. The Model Performance page builds user confidence by displaying transparent AI accuracy metrics.

## 3.3 ECONOMICAL FEASIBILITY

The entire system uses free and open-source tools with zero licensing costs. The UNSW-NB15 dataset is freely available for academic use from the University of New South Wales. The VirusTotal API free tier provides 500 URL requests per day, and Google Safe Browsing API free tier provides 10,000 requests per day both quantities are more than sufficient for academic demonstration and testing purposes.

The project runs on any standard personal computer, requiring no specialized hardware investment. Development was completed using only freely available tools including Python, VS Code, and standard Python libraries. There are no recurring costs associated with running the system for academic purposes. If deployed in a small organization, the only potential cost would be upgrading to a paid VirusTotal API tier for higher request volumes, which is optional.

### 3.4 OPERATIONAL FEASIBILITY

The dashboard is designed with usability as a primary objective. The Streamlit interface provides intuitive sidebar navigation across all 7 modules, allowing users to switch between NIDS monitoring, HIDS alerts, and web scanning with a single click. Users can start monitoring with a single button press, enter URLs for scanning via simple text fields, and configure email alerts through standard form inputs.

The system provides clear, color-coded results for green for normal or clean traffic, and red for attack or threat detections making it accessible to both technical and non-technical users. Alert logs include timestamp, severity level, category, and detailed description for easy interpretation. The manual packet predict feature allows users to demonstrate the system's detection capability without requiring live attack traffic, making it ideal for academic presentations and viva examinations.

The entire system can be started using a single command (streamlit run app.py) after activating the virtual environment, making day-to-day operation straightforward even for users without deep technical expertise.

# 4. LANGUAGE SPECIFICATION

## 4.1 OVERVIEW OF WINDOW 11

**Operating System:** Windows 11

**Description:**

Windows 11 is the latest version of Windows operating system developed by Microsoft. It serves as the primary operating environment for the Real-Time Hybrid Intrusion Detection System. Windows 11 was selected because it provides enhanced security event logging through the Windows Security Event Log, which is used by the HIDS Login Monitor module to detect failed login attempts using Event ID 4625. It also supports Administrator-level raw socket access required by Scapy for live packet capture and provides full compatibility with Python 3.12 and all required libraries used in this project.

## 4.2 OVERVIEW OF FRONTEND TOOL

**Front-End Technology:** Streamlit 1.35.0

**Description:**

The frontend of the Real-Time Hybrid Intrusion Detection System is built using Streamlit, an open-source Python framework that creates interactive web dashboard applications using pure Python code without requiring any HTML, CSS, or JavaScript knowledge. Streamlit renders all 7 dashboard pages Home, Live Monitor, Manual Predict, Model Performance, Web Threat Scanner, HIDS Monitor, and Alert Settings directly in the web browser at localhost:8501.

The dashboard uses custom CSS injected via st.markdown() to apply a dark cybersecurity-themed interface with the Orbitron and Share Tech Mono fonts for a professional appearance. Interactive elements including number inputs, checkboxes, toggle switches, buttons, tabs, and expanders are all provided by Streamlit's built-in widget library. Plotly Express and Plotly Graph Objects are integrated with Streamlit using st.plotly_chart() to render interactive pie charts, bar charts, and radar charts inside the dashboard pages.

Streamlit's session state (st.session_state) is used to maintain live packet logs, total packet and attack counters, and email configuration across page navigations without requiring a database. The @st.cache_resource decorator ensures the trained AI model is loaded from disk only once and reused across all user interactions, improving dashboard performance.

### 4.3 OVERVIEW OF BACKEND TOOL

**Back-End Technology:** Python 3.12, Scikit-learn, Scapy, Watchdog, Psutil

**Description:**

The backend of the Real-Time Hybrid Intrusion Detection System is powered entirely by Python 3.12 as the core server-side language. Python handles all data processing, machine learning, packet capture, host monitoring, API communication, and email alerting operations that run behind the Streamlit frontend.

**Machine Learning Engine:** Scikit-learn

The machine learning backend uses scikit-learn's RandomForestClassifier as the primary AI engine. The model is trained on 50,000 records sampled from the UNSW-NB15 benchmark dataset using an 80/20 train-test split with stratified sampling. The trained model, label encoder, feature column list, and evaluation metrics are all serialized to disk using the joblib library as .pkl files and reloaded when the dashboard starts.

**Network Packet Capture:** Scapy

Scapy serves as the network-level backend engine for the NIDS layer. The sniff() function captures raw packets from the host network interface in real time. Each packet is parsed to extract 9 numerical features protocol, source port, destination port, packet length, TTL, IHL, TOS, fragment offset, and TCP flags which are passed to the trained model for classification.

**Host Monitoring:** Watchdog and Psutil

The HIDS backend uses the watchdog library's Observer class to monitor file system events in the Documents and Desktop directories continuously. The psutil library scans all running system

processes every 10 seconds for known hacking tools and abnormal resource usage. Windows Security Event Log is queried every 30 seconds using the wevtutil command for failed login attempts.

**Authentication and Security:** VirusTotal API v3, Google Safe Browsing API v4

Web threat detection is handled by authenticated API calls using the requests library. VirusTotal API keys and Google Safe Browsing API keys are stored securely in config.py. All API requests use HTTPS connections. File scanning uses locally computed SHA-256 hashes via Python's hashlib module so that actual file contents are never transmitted to external servers.

**Other Tools and Libraries:**

- Threading for running HIDS monitoring and email alerts as background daemon threads without blocking the main dashboard thread.
- Smtplib and email.mime for constructing and sending HTML-formatted attack alert emails via Gmail SMTP on port 587 using STARTTLS encryption.
- Pandas and NumPy for dataset reading, feature mapping, and array construction throughout the data pipeline.
- Plotly for generating interactive data visualizations rendered inside the Streamlit frontend.
- Hashlib for computing SHA-256 file fingerprints used in the malware file scanning feature.

**Deployment:**

The application is deployed locally on a Windows 11 machine and accessed via the browser at http://localhost:8501. The Streamlit server is started using the command streamlit run app.py from the activated Python virtual environment. The project uses uv for fast virtual environment creation and dependency installation. For future deployment, the system can be containerized using Docker and hosted on cloud platforms such as AWS, Azure, or Google Cloud Platform to enable network-wide monitoring across distributed systems.

# 5. SYSTEM DESIGN

## 5.1 FILE DESIGN

**File Structure:**

**Model and Training Files:**

- **UNSW_NB15_training-set.csv:** Source dataset with 175,341 labeled network traffic records (9 attack categories), placed in the dataset/ directory.
- **capture.py:** Reads the UNSW-NB15 CSV, maps columns to 9 feature fields, samples 50,000 balanced records, and outputs live_traffic.csv.
- **live_traffic.csv:** Generated training file with 50,000 records - 9 feature columns (protocol, src_port, dst_port, pkt_len, ttl, ihl, tos, frag_offset, tcp_flags) plus binary label.
- **train_model.py:** Trains Random Forest and Extra Trees classifiers, evaluates both, saves the best-performing model and all PKL output files.

**Serialised Model Files (PKL):**

- **ids_model.pkl:** Trained classifier saved via joblib.dump(). Loaded at dashboard startup for Live Monitor and Manual Predict pages.
- **feature_cols.pkl:** Serialised list of 9 feature column names ensuring consistent ordering between training and live prediction.
- **label_encoder.pkl:** Scikit-learn LabelEncoder mapping normal=0, attack=1 for model output conversion.
- **model_metrics.pkl:** Dictionary of all evaluation results - accuracy, precision, recall, F1 score loaded by the Model Performance page.
- **confusion_matrix.png:** Heatmap image of True Normal, True Attack, False Positive, and False Negative counts from the 20% test set.

**Dashboard Application Files:**

- **app.py:** Main Streamlit application containing all 7 dashboard pages accessed through sidebar navigation.
- **web_threat_detector.py:** Module with functions for URL and file scanning via VirusTotal API v3 and Google Safe Browsing API v4.
- **hids_monitor.py:** Module implementing the three HIDS background monitoring threads, File Integrity, Process Monitor, and Login Monitor.
- **config.py:** Stores VirusTotal and Google Safe Browsing API keys as constants. Imported by web_threat_detector.py.

**Directory Structure:**

- **dataset/:** Stores the UNSW-NB15 source CSV file.
- **models/:** Stores all generated PKL files and confusion_matrix.png after training.
- **assets/:** Stores static image assets used by the dashboard.
- **requirements.txt:** Lists all Python library dependencies for reproducible environment setup using pip or uv.

## 5.2 INPUT DESIGN

**Live Packet Capture Input (NIDS Layer):**

- Captures raw network packets using Scapy's sniff () function from the active network interface in real time, with a user-configurable packet count (10 - 500) via a slider and a 30-second maximum timeout.
- Automatically filters out non-IP frames (ARP, Ethernet-only) to retain only IP-layer packets containing the required 9 feature fields.

**Packet Feature Extraction Mechanism:**

- Extracts nine numerical features from each packet: protocol, source port, destination port, packet length, TTL, IHL, TOS, fragment offset, and TCP flags.
- Constructs a single-row NumPy feature array in the exact column order used during training. Assigns default value 0 to any missing field (e.g., TCP fields in UDP/ICMP packets).

**Manual Packet Feature Input (Manual Predict Page):**

- Provides nine numeric input fields - one per feature with validation ensuring non-negative integers within valid ranges (protocol 0–255, ports 0–65535, TTL 0–255).
- Includes pre-set guidance values for normal traffic (Protocol=6, Port=443, TTL=64, PktLen=1460, TCPFlags=16) and attack traffic (Protocol=6, Port=0, TTL=1, PktLen=40, TCPFlags=0) to assist with demonstration.

**URL and File Input (Web Threat Scanner):**

- Accepts a URL string via a text input field; validates it is non-empty before calling VirusTotal and Google Safe Browsing APIs. Accepts a file path and validates using os.path.exists() before processing.
- Computes the SHA-256 hash of the file locally using Python's hashlib module and submits only the hash to VirusTotal, the actual file never leaves the system.

**Email Alert Configuration Input (Alert Settings Page):**

- Accept sender Gmail address, 16-character Gmail App Password (masked), and recipient email address. Validates the App Password length before enabling the test email button.
- Includes a boolean toggle switch to enable or disable email alerts globally across all three monitoring layers NIDS, HIDS, and Web Threat Detection.

**Host Monitoring Input (HIDS Layer):**

- A START HIDS button activates three background daemon threads simultaneously: File Integrity Monitor (watchdog - watches Desktop and Documents), Process Monitor (psutil - scans process every 10 seconds), and Login Monitor (subprocess - queries Windows Event Log for Event ID 4625 every 30 seconds).

## 5.3 OUTPUT DESIGN

**Live Packet Classification Log Output:**

- Displays a scrollable real-time log table with columns for timestamp, source IP, destination IP, protocol, packet length, label, and confidence percentage. Green rows indicate NORMAL; red rows indicate ATTACK.

- Shows a real-time Plotly pie chart below the log displaying the proportion of normal versus attack packets in the current capture session.

**Manual Predict Result Output:**

- Displays a GREEN badge (NORMAL) or RED badge (ATTACK) with a confidence percentage metric card. An email alert section automatically expands below the result only when an ATTACK is detected.

**Model Performance Output:**

- Displays four metric cards Accuracy, Precision, Recall, F1 Score at the top of the page. An interactive bar chart compares Random Forest vs Extra Trees accuracy; a radar chart visualises all four metrics simultaneously.

- Shows the confusion matrix PNG heatmap (True Normal, True Attack, False Positive, False Negative) and a full per-class classification report table at the bottom of the page.

**Web Threat Scanner Output:**

- Displays a GREEN CLEAN badge with engine statistics when no threat is found, or a RED THREAT DETECTED badge showing malicious engine count and Google Safe Browsing threat type (MALWARE, SOCIAL_ENGINEERING, PHISHING) when flagged.

- Displays the SHA-256 hash alongside the VirusTotal verdict for file scans, confirming only the hash was transmitted.

**HIDS Monitor Alert Output:**

- Displays alerts in a chronological log with color-coded severity borders red (HIGH), yellow (MEDIUM), green (LOW) showing timestamp, category (File Integrity / Process Monitor / Login Monitor), message, and contextual details.

- Provides a STOP HIDS button to terminate all monitoring threads and a CLEAR ALERTS button to reset the log.

**Email Alert Output:**

- Sends an HTML-formatted email via Gmail SMTP (port 587, STARTTLS) containing alert type, timestamp, source IP or URL, destination IP, and confidence percentage whenever an ATTACK, HIGH HIDS alert, or THREAT DETECTED result is raised.

- Email is dispatched in a background daemon thread to keep the dashboard fully responsive during transmission.

## 5.4 CODE DESIGN

**Dataset Preparation Module (capture.py):**

- Reads the UNSW-NB15 CSV using Pandas, maps 49 original columns to 9 standard packet features, handles missing values with fillna(0), samples 50,000 records using random_state=42, and exports live_traffic.csv with binary labels (normal=0, attack=1).

**AI Model Training Module (train_model.py):**

- Trains RandomForestClassifier and ExtraTreesClassifier (100 trees each, n_jobs=-1) on an 80/20 stratified split. Evaluates both using accuracy_score(), precision_score(), recall_score(), f1_score (), and confusion_matrix().

- Saves the better-performing model as ids_model.pkl using joblib.dump() along with feature_cols.pkl, label_encoder.pkl, model_metrics.pkl, and confusion_matrix.png.

**NIDS Live Detection Module (app.py - Live Monitor):**

- Runs Scapy's sniff () in a separate thread with a configurable count and 30-second timeout. Extracts 9 features from each IP packet, constructs a NumPy array, and passes it to ids_model.pkl for classification.

- Retrieves predicted label and confidence from model.predict() and model.predict_proba(). Appends result to st.session_state.live_log and triggers email alert in a background thread if ATTACK is detected.

**Web Threat Detection Module (web_threat_detector.py):**

- check_url_virustotal() base64-encodes the URL and sends a GET request to VirusTotal API v3; check_google_safe_browsing() sends a POST request with MALWARE and SOCIAL_ENGINEERING filters to Safe Browsing API v4.

- check_file_virustotal() computes the SHA-256 hash locally using hashlib. sha256() and submits only the hash. All calls use try-except with a 10-second timeout to prevent dashboard freezing.

**HIDS Monitoring Module (hids_monitor.py):**

- FileIntegrityMonitor extends watchdog's FileSystemEventHandler, raising HIGH alerts for .exe, .bat, .ps1, .dll, .vbs files. ProcessMonitor calls psutil.process_iter() every 10 seconds against a list of known tools (Mimikatz, Netcat, Nmap, Metasploit). LoginMonitor queries Event ID 4625 through subprocess every 30 seconds.

- All three monitors launch as daemon threads using threading.Thread(daemon=True), terminating automatically when the main dashboard exits.

**Dashboard and User Interface (app.py):**

- Uses st.sidebar with st.radio() for navigation. Applies @st.cache_resource to load all PKL files once at startup. Manages packet logs, HIDS alerts, and email config via st.session_state across all page navigations.

- Injects custom CSS via st.markdown(unsafe_allow_html=True) for color-coded alert borders (HIGH=red, MEDIUM=yellow, LOW=green) and dark-theme compatible label visibility.

**Email Alert System (app.py - Alert Settings):**

- send_alert() connects to smtp.gmail.com on port 587 using smtplib.SMTP(), calls starttls() for TLS encryption, and sends an HTML-formatted message built with MIMEMultipart and MIMEText containing full threat details.

- Validates the 16-character Gmail App Password and confirms SMTP connectivity through a test email button before enabling live alerts across all three detection layers.

# 6. MODULES

**Module 1: Home Dashboard**

The Home page serves as the central command center and landing page for the entire Hybrid IDS system. It is the first page users to see when the Streamlit dashboard loads and provides a comprehensive overview of the system's current state and capability.

The page displays the system title 'HYBRID INTRUSION DETECTION SYSTEM' prominently at the top, followed by four metric cards showing the trained model's Accuracy, Precision, Recall, and F1 Score values loaded from model_metrics.pkl. Three colored description cards introduce the three detection layers (NIDS, HIDS, Web Threats) with icons and brief explanations. A visual list of all detectable attack types with color indicators is shown below, followed by a system pipeline diagram illustrating the 5-step flow from data capture to alert generation.

**Module 2: Live Network Monitor (NIDS)**

The Live Network Monitor is the core NIDS module and the most technically significant page of the dashboard. It performs real-time network packet capture and AI-powered threat classification simultaneously.

The user first configures the packet capture count (between 10 and 500 packets) using a slider, and optionally enables email alerts for any detected attack. Upon clicking the CAPTURE & ANALYSE button, Scapy begins capturing live network packets from the active interface with a 30-second capture timeout. For each captured packet, the 9 standard features are extracted: protocol number, source port, destination port, packet length, TTL, IP header length, TOS field, fragment offset, and TCP flags value.

These 9 features are passed to the loaded Random Forest model (ids_model.pkl), which produces an instantaneous NORMAL or ATTACK classification along with a confidence percentage. Results are displayed in a color-coded live log (red for attack, green for normal) showing timestamp, source IP,

destination IP, protocol, packet length, label, and confidence. A Plotly pie chart at the bottom summarizes the traffic distribution between normal and attack packets.

**Module 3: Manual Packet Predict**

The Manual Packet Predict module allows users to manually input all 9 packet feature values and receive an instant AI classification. This module is particularly valuable for demonstration purposes during academic presentations and viva examinations, as it allows reliable demonstration of attack detection without requiring actual malicious traffic on the network.

The page presents 9 labeled input fields corresponding to each packet feature. The user can enter normal browsing values (Protocol=6, SrcPort=52000, DstPort=443, PktLen=1460, TTL=64, IHL=5, TOS=0, FragOffset=0, TCPFlags=16) to receive a GREEN NORMAL badge, or attack values (Protocol=6, SrcPort=0, DstPort=0, PktLen=40, TTL=1, IHL=5, TOS=0, FragOffset=0, TCPFlags=0) to receive a RED ATTACK badge with the confidence percentage. When an attack is detected, an expandable email alert section appears below the result.

**Module 4: Model Performance**

The Model Performance module provides complete transparency into the AI model's evaluation metrics, enabling users to verify the system's effectiveness and understand how well it performs on unseen data from the UNSW-NB15 test set.

The page displays four metric cards showing Accuracy, Precision, Recall, and F1 Score loaded from model_metrics.pkl. An interactive Plotly bar chart compares the performance of the Random Forest classifier against the Extra Trees classifier trained during the same session. A radar chart visualizes all four metrics simultaneously for a holistic view of model quality. The confusion matrix image (confusion_matrix.png) is displayed showing the counts of True Normal, True Attack, False Positive (normal classified as attack), and False Negative (attack classified as normal) predictions. A per-class classification report provides the complete breakdown of performance by class.

**Module 5: Web Threat Scanner**

The Web Threat Scanner module enables on-demand scanning of URLs and files against two industry-standard threat intelligence databases, providing web-layer security that complements the network and host monitoring layers.

The module is divided into two tabs. The URL Check tab allows users to enter any web address for scanning. The system simultaneously queries the VirusTotal API (checking the URL against 70+ antivirus engine databases) and the Google Safe Browsing API (checking against Google's database of billions of known phishing, malware, and social engineering sites). Results are returned within seconds and displayed as either a green CLEAN badge with engine statistics or a red THREAT DETECTED badge with details about which engines flagged the URL and for what reason.

The File Check tab enables scanning of any local file. The system computes the file's SHA-256 cryptographic hash locally using Python's hashlib library and sends only this hash to VirusTotal for lookup. This means the actual file contents never leave the user's machine, protecting sensitive documents from unnecessary exposure. The result shows whether VirusTotal's database has previously analyzed this file and whether it was flagged as malicious.

**Module 6: HIDS Monitor**

The HIDS Monitor module provides continuous real-time monitoring of the host machine's file system, running processes, and login activity. It operates as three independent background threads that run simultaneously without blocking the Streamlit interface.

When the user clicks the START HIDS button, three monitoring threads activate: the File Integrity Monitor uses the watchdog library to watch the Documents and Desktop directories for any file creation, modification, deletion, or renaming events. Files with suspicious extensions (.exe, .bat, .ps1, .dll, .vbs) trigger HIGH severity alerts immediately. The Process Monitor uses psutil to enumerate all running processes every 10 seconds, checking each process name against a list of known hacking tools including Mimikatz, Netcat, Nmap, Metasploit, and others.

The Login Monitor queries the Windows Security Event Log every 30 seconds for Event ID 4625 (failed login attempt), and multiple failures within a short period trigger a brute-force attack alert.

All alerts are displayed in the dashboard with color-coded severity borders (red for HIGH, yellow for MEDIUM, green for LOW). The user can filter alerts by category and clear the alert log between monitoring sessions.

**Module 7: Alert Settings**

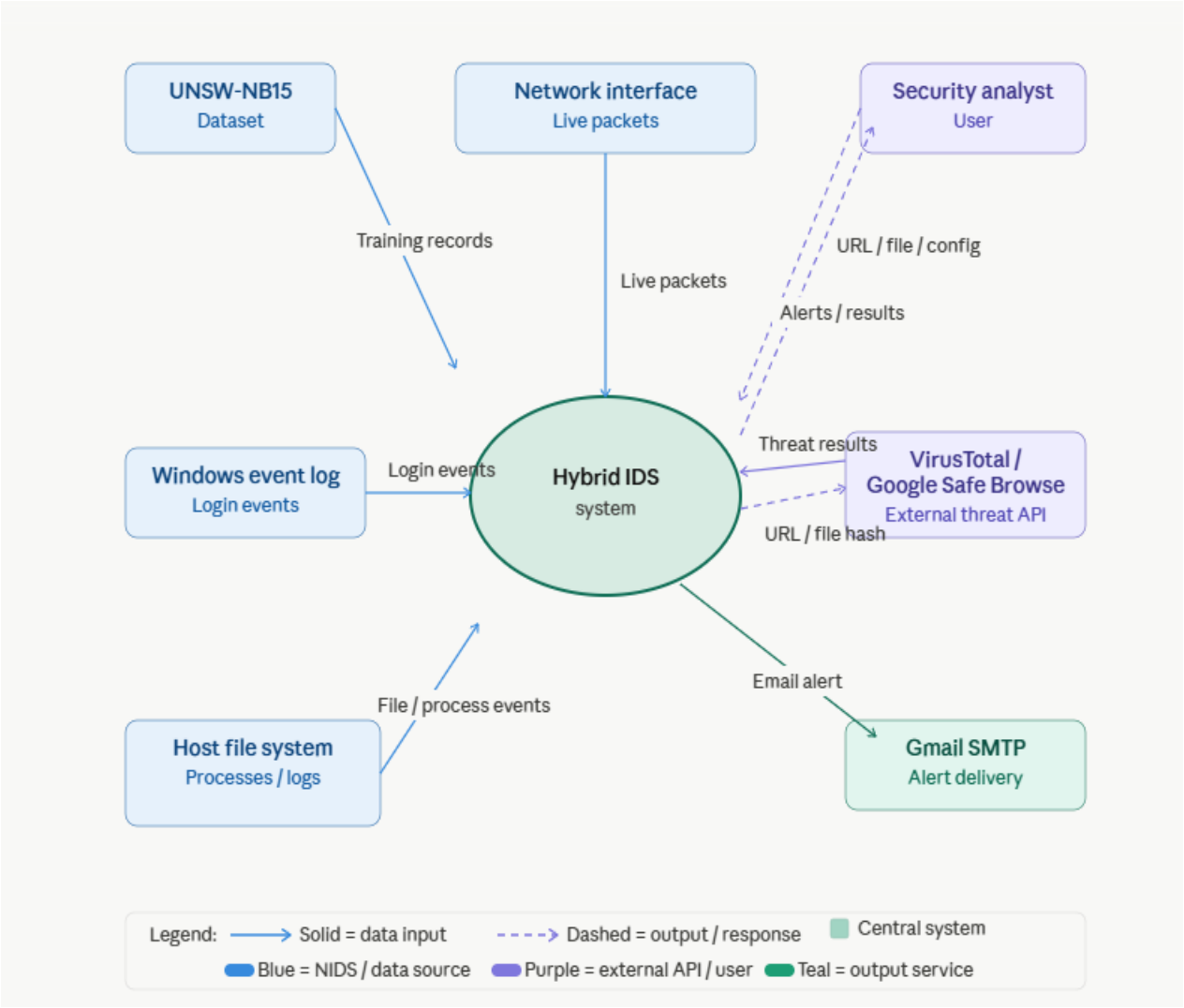
The Alert Settings module provides configuration options for the automated email notification system, allowing the security analyst to receive real-time alerts on their email when any threat is detected across any monitoring layer.

The user enters their Gmail address, their Gmail App Password (a 16-character application-specific password generated from Google Account Security settings), and the recipient email address. A test email button verifies the SMTP connection before enabling live alerts. A toggle switch enables or disables email notifications globally across all three detection layers. When enabled, any ATTACK detection in the Live Monitor, any HIGH severity alert from HIDS, or any THREAT DETECTED result from the Web Scanner triggers an automated email through Gmail SMTP.

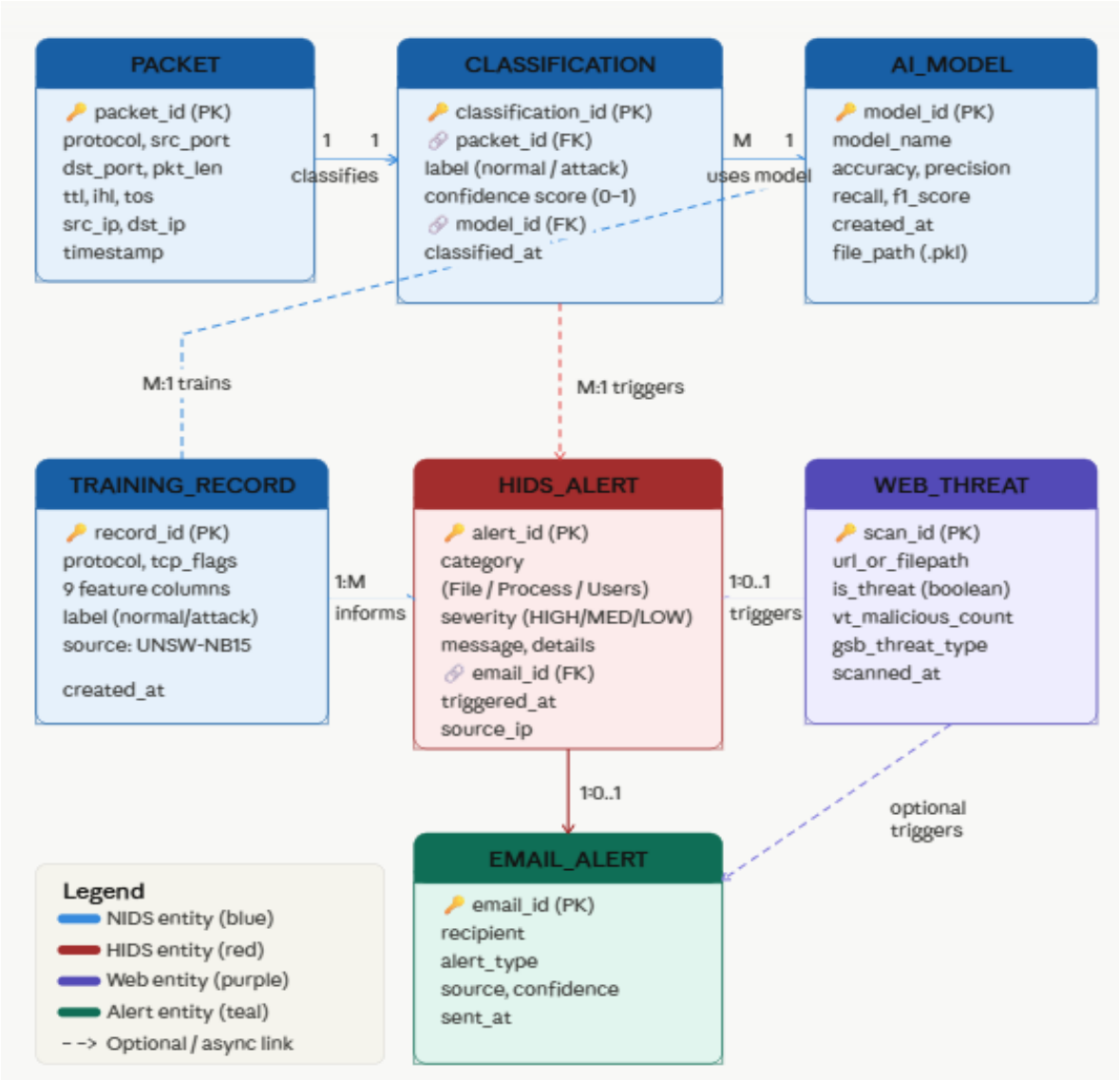
# 7. DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) illustrates how data moves through the Hybrid IDS showing all external entities, internal processes, data stores, and the flow of data between them.

Symbols used: Rectangle = External Entity | Circle/Oval = Process | Arrow = Data Flow



# 8. ER DIAGRAM


PACKET
packet_id (PK)
protocol, src_port
dst_port, pkt_len
ttl, ihl, tos
src_ip, dst_ip
timestamp
1
1
classifies
CLASSIFICATION
classification_id (PK)
packet_id (FK)
label (normal / attack)
confidence score (0–1)
model_id (FK)
classified_at
M
1
uses model
AI_MODEL
model_id (PK)
model_name
accuracy, precision
recall, f1_score
created_at
file_path (.pkl)
M:1 trains
M:1 triggers
TRAINING_RECORD
record_id (PK)
protocol, tcp_flags
9 feature columns
label (normal/attack)
source: UNSW-NB15
created_at
1:M
informs
HIDS_ALERT
alert_id (PK)
category
(File / Process / Users)
severity (HIGH/MED/LOW)
message, details
email_id (FK)
triggered_at
source_ip
1:0..1
triggers
WEB_THREAT
scan_id (PK)
url_or_filepath
is_threat (boolean)
vt_malicious_count
gsb_threat_type
scanned_at
1:0..1
optional
triggers
EMAIL_ALERT
email_id (PK)
recipient
alert_type
source, confidence
sent_at
Legend
NIDS entity (blue)
HIDS entity (red)
Web entity (purple)
Alert entity (teal)
--> Optional / async link

**Key Points:**

**Entities:**

- **Packet:** Represents a single network packet captured from the live network interface by Scapy during a monitoring session.
- **Classification:** Represents the AI model's prediction result for each packet, containing the label (normal/attack) and confidence score.
- **Training Record:** Represents a single labeled record from the UNSW-NB15 dataset used to train the Random Forest model.
- **AI Model:** Represents the trained Random Forest classifier saved as ids_model.pkl and used for all packet predictions.
- **HIDS Alert:** Represents an alert generated by the HIDS module from file monitoring, process scanning, or login event detection.
- **Web Scan:** Represents the result of a URL or file scan performed using VirusTotal API or Google Safe Browsing API.
- **Email Alert:** Represents an automated notification email sent when any threat is detected across any detection layer.
- **User Config:** Represents the dashboard settings entered by the user including email credentials and alert preferences.

**Relationships:**

- **Packet - Classification:** One-to-One - each packet receives exactly one AI classification result.
- **Training Record - AI Model:** Many-to-One - many UNSW-NB15 records are used to train one AI model.
- **AI Model - Packet:** One-to-Many - one trained model classifies many live packets during monitoring.
- **Classification - Email Alert:** One-to-Zero-or-One - an email is triggered only when the prediction label is ATTACK.
- **HIDS Alert - Email Alert:** One-to-Zero-or-One - an email is triggered only for HIGH severity HIDS alerts.

- **Web Scan Email Alert:** One-to-Zero-or-One - an email is triggered only when a threat is detected.
- **User Config - Email Alert:** One-to-Many - one configuration applies to all alerts across all three layers.

**Attributes:**

- **Packet:** Packet_ID (PK), protocol, src_ip, dst_ip, src_port, dst_port, pkt_len, ttl, ihl, tos, frag_offset, tcp_flags, captured_at.
- **Classification:** Classification_ID (PK), Packet_ID (FK), label, confidence_percent, classified_at.
- **Training Record:** Record_ID (PK), protocol, src_port, dst_port, pkt_len, ttl, ihl, tos, frag_offset, tcp_flags, label.
- **AI Model:** Model_ID (PK), model_name, accuracy, precision, recall, f1_score, created_at, file_path.
- **HIDS Alert:** Alert_ID (PK), category, severity, message, details, triggered_at.
- **Web Scan:** Scan_ID (PK), url_or_filepath, scan_type, is_threat, vt_malicious_count, gsb_threat_type, scanned_at.
- **Email Alert:** Email_ID (PK), recipient_email, alert_type, source, confidence, sent_at.
- **User Config:** Config_ID (PK), sender_email, receiver_email, email_alerts_enabled, packets_per_capture.

**Constraints:**

- **Unique Constraints:** All primary keys - Packet_ID, Classification_ID, Alert_ID, Scan_ID, Email_ID - must be unique across their entities.
- **Foreign Key Constraints:** Classification references Packet through Packet_ID. Email Alert references Classification, HIDS Alert, and Web Scan through their respective primary keys.
- **Not Null Constraints:** Fields such as label, severity, is_threat, and recipient_email must always contain a value and cannot be null.
- **Check Constraints:** The label field must only contain 'normal' or 'attack'. The severity field

must only contain 'HIGH', 'MEDIUM', or 'LOW'.

**Cardinalities:**

- **Packet Classification:** One-to-One - each packet is classified exactly once.
- **Training Record - AI Model:** Many-to-One - 50,000 records train one single model.
- **AI Model - Packet:** One-to-Many - one model classifies unlimited live packets.
- **Classification - Email Alert:** One-to-Zero-or-One - only attack predictions trigger email.
- **HIDS Alert - Email Alert:** One-to-Zero-or-One - only HIGH alerts trigger email.
- **User Config - Email Alert:** One-to-Many - one config applies to all outgoing alerts.

**Normalization:**

The ER diagram follows standard normalization principles. First Normal Form (1NF) is achieved by storing all packet features as individual atomic columns. Second Normal Form (2NF) is achieved by ensuring all attributes are fully dependent on their primary key. Third Normal Form (3NF) is achieved by removing transitive dependencies model metrics are stored in the AI Model entity and not repeated inside Classification.

**Additional Features:**

- The last 200 packet classifications are maintained in Streamlit session state as a temporary runtime log during active monitoring.
- Future enhancement can include a persistent SQLite database to store all alerts and scan results permanently for forensic audit trail purposes.
- Additional entities such as Network Session, Threat Report, and User Activity Log can be introduced in future versions of the system.

# 9. SYSTEM TESTING AND MAINTENANCE

## SYSTEM TESTING

### Unit Testing:

Individual components of the Hybrid IDS such as packet feature extraction, AI model prediction, web threat API calls, and HIDS monitoring functions are tested in isolation to verify their correct behavior before integration.

### Integration Testing:

Once individual modules are tested, integration testing verifies that capture.py output feeds correctly into train_model.py, the trained model loads properly in the dashboard, and all three detection layers NIDS, HIDS, and Web Threat  operate together without conflicts.

### End-to-End Testing:

End-to-end testing simulates real usage scenarios including live packet capture and classification, manual packet prediction with both normal and attack values, URL scanning using VirusTotal and Google Safe Browsing, HIDS file integrity triggering, and email alert delivery.

### Performance Testing:

Performance testing assesses the system's responsiveness under load conditions. This includes verifying that the Random Forest model trains within 15 seconds on 50,000 records, live packet capture completes within 30 seconds, and single packet prediction responds in under 5 milliseconds.

### Security Testing:

Security testing verifies that API keys stored in config.py are not exposed in dashboard output, file scanning sends only SHA-256 hashes without uploading file contents, and email transmission uses STARTTLS encryption over Gmail SMTP port 587.

## MAINTENANCE

### Bug Fixing:

Regular monitoring and user feedback help identify issues in the system. Maintenance involves promptly fixing errors in packet parsing, API response handling, or dashboard rendering through code patches and library updates.

### Performance Optimization:

Continuous monitoring identifies bottlenecks such as slow model loading or delayed API responses. Maintenance involves applying @st.cache_resource caching, reducing packet batch sizes, and optimizing API timeout settings.

### Security Updates:

As new threats emerge, the system requires regular updates to API keys, Python library versions, and HIDS suspicious process lists to stay current with the evolving cybersecurity landscape.

### Feature Enhancements:

Based on user feedback, the system can be enhanced with DNS monitoring, automatic IP blocking, persistent database logging, and mobile push notification support as planned future enhancements.

# 10. SYSTEM IMPLEMENTATION

## ENVIRONMENT SETUP

Install Python 3.12 from the official Python website and verify the installation using python --version in PowerShell.

Install the uv package manager using pip install uv and create a virtual environment using uv venv --python 3.12.

Set the PowerShell execution policy using Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser to allow script execution.

Activate the virtual environment using .venv\Scripts\activate and install all dependencies using uv pip to install -r requirements.txt.

## DATASET AND API CONFIGURATION

Create a folder named dataset inside the project directory and place the UNSW_NB15_training-set.csv file inside it.

Register for a free VirusTotal account at virustotal.com and obtain the API key from the account profile page.

Enable the Google Safe Browsing API in Google Cloud Console and obtain the API key from the credentials section.

Open config.py and replace the placeholder values with the actual VirusTotal and Google Safe Browsing API keys.

## MODEL TRAINING

Open PowerShell as Administrator, navigate to the project folder, and activate the virtual environment.

Run python capture.py to read the UNSW-NB15 dataset and generate live_traffic.csv with 50,000 labeled records containing both normal and attack traffic.

Run python train_model.py to train the Random Forest and Extra Trees classifiers, evaluate both models, and save ids_model.pkl, feature_cols.pkl, label_encoder.pkl, model_metrics.pkl, and confusion_matrix.png.

Note the accuracy, precision, recall, and F1 score values displayed in the terminal after training completes.

## TESTING AND QUALITY ASSURANCE

Run streamlit run app.py and open http://localhost:8501 in the browser to verify the dashboard loads with correct model metrics.

Navigate to Manual Predict and test with normal values (Protocol=6, Port=443, TTL=64) to verify a NORMAL result, then test with attack values (Protocol=6, Port=0, TTL=1) to verify an ATTACK result.

Navigate to Web Threat Scanner and scan https://www.google.com to verify a CLEAN result is returned correctly.

Navigate to HIDS Monitor, click START HIDS, create a .exe file on the Desktop, and verify a HIGH severity File Integrity alert appears within 5 seconds.

Navigate to Alert Settings, enter Gmail credentials, and click SEND TEST EMAIL to verify email alert delivery.

## DEPLOYMENT AND ROLLOUT

Deploy the system locally on a Windows 11 machine by running streamlit run app.py from the activated virtual environment. The dashboard is accessible at http://localhost:8501.

For network-wide deployment, the application can be containerized using Docker and hosted on cloud platforms such as AWS, Azure, or Google Cloud Platform to enable monitoring across distributed systems.

Provide a 30-minute walkthrough to new users covering all 7 dashboard pages, demonstrating Live Monitor, Manual Predict, Web Threat Scanner, and HIDS Monitor operations.

## MONITORING AND MAINTENANCE

Monitor the dashboard's Total Packets, Total Attacks, and Attack Rate metrics regularly to identify unusual spikes in network activity.

Respond to HIDS HIGH severity alerts immediately by investigating the flagged file path, process name, or failed login count for potential security incidents.

Conduct quarterly model retraining using updated UNSW-NB15 data or newer datasets to keep the AI model current with evolving attack patterns.

Perform regular API key rotation for VirusTotal and Google Safe Browsing and update Python library dependencies to patch any security vulnerabilities.

# 11. CONCLUSION AND FUTURE ENHANCEMENT

## CONCLUSION:

The development and implementation of the Real-Time Hybrid Intrusion Detection System Using Machine Learning represent a significant step forward in making professional-grade cybersecurity monitoring accessible and affordable. By integrating three complementary detection layers a Random Forest AI-powered NIDS trained on the UNSW-NB15 benchmark dataset, a Host-based IDS monitoring file integrity and processes, and a Web Threat Detection module using VirusTotal and Google Safe Browsing APIs the system provides comprehensive 360-degree security coverage that no single-layer tool can achieve. The unified Streamlit dashboard with real-time alerts, interactive charts, and email notifications makes the system practical and usable for security analysts, system administrators, and researchers alike. Built entirely using free and open-source tools, the project demonstrates that effective cybersecurity monitoring does not require expensive enterprise solutions.

## FUTURE ENHANCEMENT:

### DNS Monitoring:

Implement DNS query sniffing using Scapy to detect DNS tunneling attacks and malicious domain queries in real time.

### Automatic IP Blocking:

Integrate Windows Firewall API to automatically block attacker IP addresses when attack packets are detected by the NIDS layer.

### Persistent Database Logging:

Introduce a SQLite or PostgreSQL database to permanently store all packet logs, HIDS alerts, and web scan results across sessions for for forensic investigation and audit trail purposes.

### Mobile Push Notifications:

Develop a companion mobile application to deliver push notifications when threats are detected, enabling immediate response from anywhere.

### Cloud Deployment:

Deploy the system on AWS, Azure, or Google Cloud Platform using Docker containers to enable network-wide monitoring across distributed and enterprise environments.

### Advanced Dataset Training:

Retrain the AI model on newer datasets such as CIC-IDS-2018 or NSL-KDD to improve detection of modern attack types not covered by UNSW-NB15.

### Anomaly Detection:

Incorporate unsupervised machine learning algorithms such as Isolation Forest or Autoencoder to detect zero-day attacks that do not match known attack patterns.

### Threat Intelligence Integration:

Integrate AbuseIPDB and Shodan APIs to check source IP addresses against known malicious IP databases during live packet classification.

# 12. TABLE DESIGN

| Component | Description |
|---|---|
| live_traffic.csv | Training data file containing 50,000 records with 9 packet features and labels (normal/attack) generated from the UNSW-NB15 dataset |
| ids_model.pkl | Serialized trained Random Forest classifier loaded by the dashboard for all live and manual packet predictions |
| feature_cols.pkl | Serialized list of 9 feature column names ensuring consistent feature ordering between training and prediction |
| label_encoder.pkl | Serialized LabelEncoder that converts normal/attack labels to 0/1 during training and back to text during prediction |
| model_metrics.pkl | Serialized dictionary storing accuracy, precision, recall, F1 score, and full classification report displayed on the Model Performance page |
| confusion_matrix.png | PNG heatmap image showing true positives, true negatives, false positives, and false negatives from the model evaluation |
| config.py | Configuration file storing VirusTotal API key and Google Safe Browsing API key used by the web threat detection module |
| Packet Log | In-memory session table maintaining the last 200 captured and classified packets with timestamp, IPs, protocol, label, and confidence |
| HIDS Alert Log | In-memory session list storing all HIDS alerts with category, severity, message, details, and timestamp for the current session |
| Web Scan Result | Temporary in-memory structure holding the combined VirusTotal and Google Safe Browsing scan result for each URL or file submitted |

# 13. SAMPLE CODING

**# capture.py - Generate training data from UNSW-NB15 datase**

```
import pandas as pd
import numpy as np
import os
DATASET_PATH = "dataset/UNSW_NB15_training-set.csv"
OUTPUT_FILE = "live_traffic.csv"
NUM_ROWS = 50000
df = pd.read_csv(DATASET_PATH)

live_df = pd.DataFrame()
proto_map = {"tcp": 6, "udp": 17, "icmp": 1}
live_df["protocol"] = df["proto"].apply(lambda x: proto_map.get(str(x).lower(), 0))
live_df["src_port"] = pd.to_numeric(df["sport"], errors="coerce").fillna(0).astype(int)
live_df["dst_port"] = pd.to_numeric(df["dport"], errors="coerce").fillna(0).astype(int)
live_df["pkt_len"] = pd.to_numeric(df["spkts"], errors="coerce").fillna(60).astype(int)
live_df["ttl"] = pd.to_numeric(df["sttl"], errors="coerce").fillna(64).astype(int)
live_df["ihl"] = pd.to_numeric(df["swin"], errors="coerce").fillna(5).astype(int)
live_df["tos"] = pd.to_numeric(df.get("tos", pd.Series([0]*len(df))),
errors="coerce").fillna(0).astype(int)
live_df["frag_offset"] = pd.to_numeric(df["smeansz"], errors="coerce").fillna(0).astype(int)
live_df["tcp_flags"] = (pd.to_numeric(df["tcprtt"],
errors="coerce").fillna(0).multiply(100).astype(int))
live_df["label"] = df["label"].apply(lambda x: "attack" if str(x).strip() in ["1", "1.0"] else "normal")
live_df = live_df.sample(n=NUM_ROWS, random_state=42).fillna(0)
live_df.to_csv(OUTPUT_FILE, index=False)
print(live_df["label"].value_counts())
```

**# train_model.py - Train Random Forest AI model**

```
import pandas as pd, joblib
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score,
 classification_report, confusion_matrix)
 import matplotlib
```

```
 matplotlib.use("Agg")
 import matplotlib.pyplot as plt
 import seaborn as sns

FEATURE_COLS = ["protocol","src_port","dst_port","pkt_len",
"ttl","ihl","tos","frag_offset","tcp_flags"
df = pd.read_csv("live_traffic.csv")
 le = LabelEncoder()
 df["label_enc"] = le.fit_transform(df["label"])

X = df[FEATURE_COLS].fillna(0)
y = df["label_enc"]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)

rfc = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rfc.fit(X_train, y_train)
rfc_score = accuracy_score(y_test, rfc.predict(X_test)) * 100

etc = ExtraTreesClassifier(n_estimators=100, random_state=42, n_jobs=-1)
etc.fit(X_train, y_train)
etc_score = accuracy_score(y_test, etc.predict(X_test)) * 100

best = rfc if rfc_score >= etc_score else etc
y_pred = best.predict(X_test)

print(f"Accuracy : {round(accuracy_score(y_test,y_pred)*100,2)}%")
print(f"Precision : {round(precision_score(y_test,y_pred,average='weighted')*100,2)}%")
print(f"Recall : {round(recall_score(y_test,y_pred,average='weighted')*100,2)}%")
print(f"F1 Score : {round(f1_score(y_test,y_pred,average='weighted')*100,2)}%")

joblib.dump(best, "ids_model.pkl")
joblib.dump(le, "label_encoder.pkl")
joblib.dump(FEATURE_COLS, "feature_cols.pkl"
```

**# web_threat_detector.py - URL and file malware scanning**

```
import requests, base64, hashlib
 from config import VIRUSTOTAL_API_KEY, SAFE_BROWSING_API_KEY

def check_url_virustotal(url):
```

```
 url_id = base64.urlsafe_b64encode(url.encode()).decode().strip("=")
 resp = requests.get(
 f"https://www.virustotal.com/api/v3/urls/{url_id}",
 headers={"x-apikey": VIRUSTOTAL_API_KEY}, timeout=10)
 if resp.status_code == 200:
 stats = resp.json()["data"]["attributes"]["last_analysis_stats"]
 return {"malicious": stats.get("malicious", 0),
 "is_threat": stats.get("malicious", 0) > 0}
 return {"is_threat": False}

def check_phishing_google(url):
 payload = {
 "client": {"clientId": "HybridIDS", "clientVersion": "1.0"},
 "threatInfo": {
 "threatTypes": ["MALWARE","SOCIAL_ENGINEERING","UNWANTED_SOFTWARE"],
 "platformTypes": ["ANY_PLATFORM"],
 "threatEntryTypes": ["URL"],
 "threatEntries": [{"url": url}]
 }
 }
 resp = requests.post(
 f"https://safebrowsing.googleapis.com/v4/threatMatches:find"
 f"?key={SAFE_BROWSING_API_KEY}", json=payload, timeout=10)
 if resp.status_code == 200:
 data = resp.json()
 is_threat = "matches" in data and len(data["matches"]) > 0
 return {"is_threat": is_threat,
 "threat_type": data["matches"][0]["threatType"]
 if is_threat else "None"}
 return {"is_threat": False}



def check_file_virustotal(filepath):
 with open(filepath, "rb") as f:
 file_hash = hashlib.sha256(f.read()).hexdigest()
 resp = requests.get(
 f"https://www.virustotal.com/api/v3/files/{file_hash}",
 headers={"x-apikey": VIRUSTOTAL_API_KEY}, timeout=10)
 if resp.status_code == 200:
 stats = resp.json()["data"]["attributes"]["last_analysis_stats"]
 return {"hash": file_hash,
```

```
 "malicious": stats.get("malicious", 0),
 "is_threat": stats.get("malicious", 0) > 0}
 return {"hash": file_hash, "is_threat": False}
```

**# hids_monitor.py - Host-based IDS monitoring**

```
import os, psutil, threading
 from datetime import datetime
 from watchdog.observers import Observer
 from watchdog.events import FileSystemEventHandler

hids_alerts = []
 SUSPICIOUS_EXT = [".exe",".bat",".cmd",".ps1",".vbs",".dll"]
 HACKING_TOOLS = ["mimikatz","netcat","nc.exe","nmap","metasploit",
 "cobaltstrike","psexec","lazagne","wce.exe"]

def add_alert(category, severity, message, details=""):
 hids_alerts.insert(0, {
 "time": datetime.now().strftime("%H:%M:%S"),
 "category": category, "severity": severity,
 "message": message, "details": details
 })

class FileChangeHandler(FileSystemEventHandler):
 def on_created(self, event):
 if event.is_directory: return
 ext = os.path.splitext(event.src_path)[1].lower()
 sev = "HIGH" if ext in SUSPICIOUS_EXT else "LOW"
 add_alert("File Integrity", sev,
 f"File created: {os.path.basename(event.src_path)}",
 f"Path: {event.src_path}")

def on_modified(self, event):
 if event.is_directory: return
 ext = os.path.splitext(event.src_path)[1].lower()
 if ext in SUSPICIOUS_EXT:
 add_alert("File Integrity","MEDIUM",
 f"Executable modified: {os.path.basename(event.src_path)}",
 f"Path: {event.src_path}")

def scan_processes():
```

```
 while True:
 for proc in psutil.process_iter(["pid","name","cpu_percent","memory_percent"]):
 try:
 name = (proc.info["name"] or "").lower()
 for tool in HACKING_TOOLS:
 if tool in name:
 add_alert("Process Monitor","HIGH",
 f"Suspicious process: {proc.info['name']}",
 f"PID: {proc.info['pid']}")
 except (psutil.NoSuchProcess, psutil.AccessDenied):
 pass
 threading.Event().wait(10)

def start_hids():
obs = Observer()
obs.schedule(FileChangeHandler(),
os.path.expanduser("~/Desktop"), recursive=True)
obs.start()
threading.Thread(target=scan_processes, daemon=True).start()

def get_hids_alerts():
 return hids_alerts.copy()
```

## Code Explaination :

### Import Statements:

- pandas, numpy, os: Used in capture.py for reading the UNSW-NB15 CSV file, mapping columns to 9 features, and saving live_traffic.csv.
- sklearn:Provides RandomForestClassifier, ExtraTreesClassifier, train_test_split, LabelEncoder, and all evaluation metric functions used in train_model.py.
- requests, base64, hashlib: Used in web_threat_detector.py for making HTTPS API calls to VirusTotal and Google Safe Browsing, encoding URLs, and computing SHA-256 file hashes.
- watchdog, psutil, threading: Used in hids_monitor.py for file system event monitoring, process scanning, and running background daemon threads.

### capture.py:

- Reads the UNSW-NB15 dataset CSV file and maps its columns to the 9 standard packet feature columns required by the AI model.
- Converts text protocol names (tcp, udp, icmp) to numeric values (6, 17, 1) using a protocol map dictionary.
- Converts the original label column to binary - '1' becomes 'attack' and '0' becomes 'normal'.
- Samples 50,000 records randomly, fills any missing values with 0, and saves the result as live_traffic.csv.

### train_model.py:

- Reads live_traffic.csv and encodes string labels to integers using LabelEncoder (normal=0, attack=1).
- Splits the data 80% for training and 20% for testing using stratified sampling to maintain class balance.
- Trains both RandomForestClassifier and ExtraTreesClassifier with 100 trees each and selects the better performing model.
- Prints Accuracy, Precision, Recall, and F1 Score on the test set and saves the best model, encoder, and feature list as PKL files.

**web_threat_detector.py:**

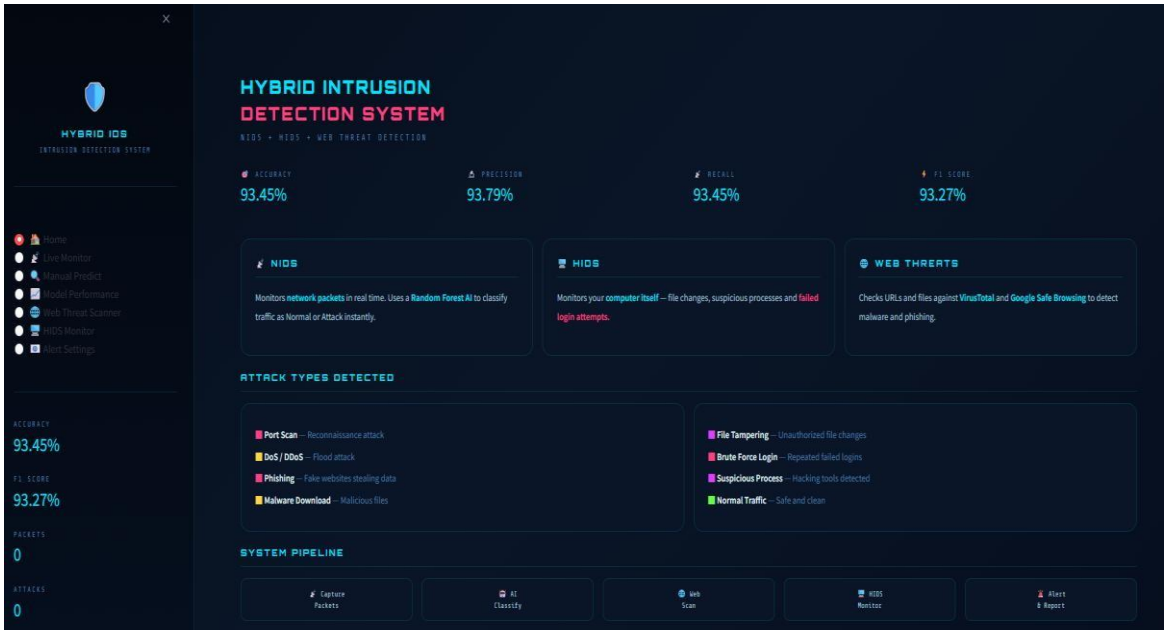
- check_url_virustotal() base64-encodes the URL, sends a GET request to VirusTotal API v3, and returns the number of malicious engine detections.
- check_phishing_google() sends a POST request to Google Safe Browsing API v4 with the URL and threat type filters, returning the threat type if found.
- check_file_virustotal() computes the file's SHA-256 hash locally and queries VirusTotal using only the hash — the actual file is never uploaded.

**hids_monitor.py:**

- FileChangeHandler class overrides on_created() and on_modified() methods to generate HIGH alerts for suspicious file extensions and MEDIUM alerts for modified executables.
- scan_processes() function runs in a background daemon thread, scanning all processes every 10 seconds and raising HIGH alerts when known hacking tool names are detected.
- start_hids() activates both the file system observer on the Desktop directory and the process scanner thread with a single function call from the dashboard.

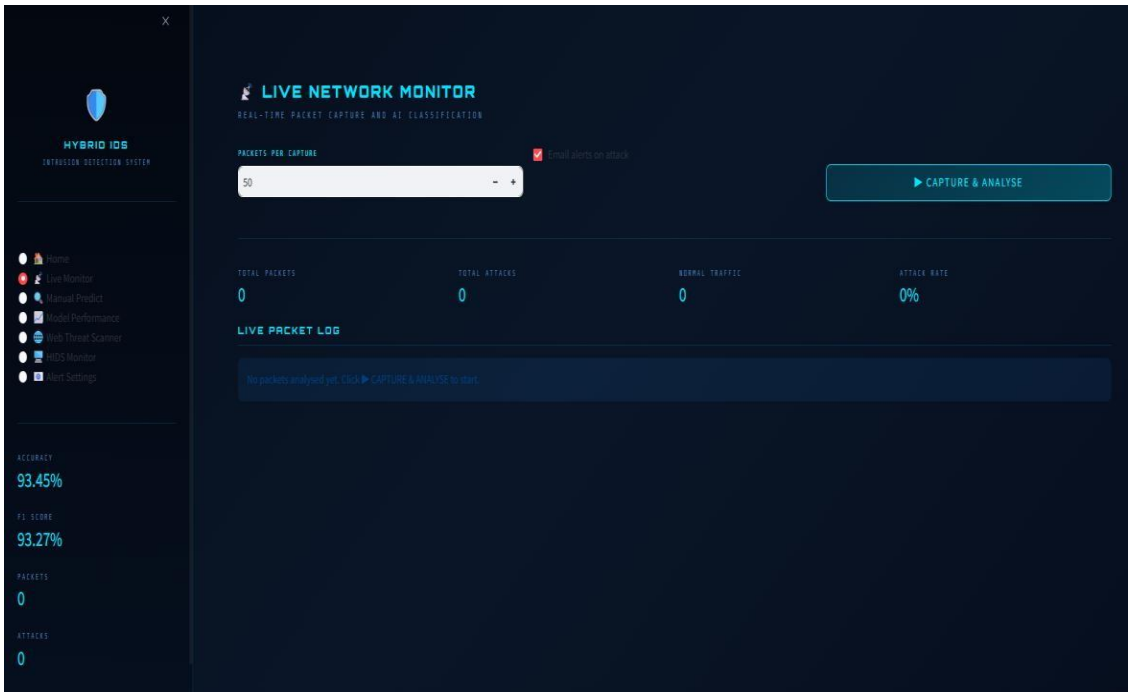
# 14. SAMPLE OUTPUT

## 9.1 Home Dashboard



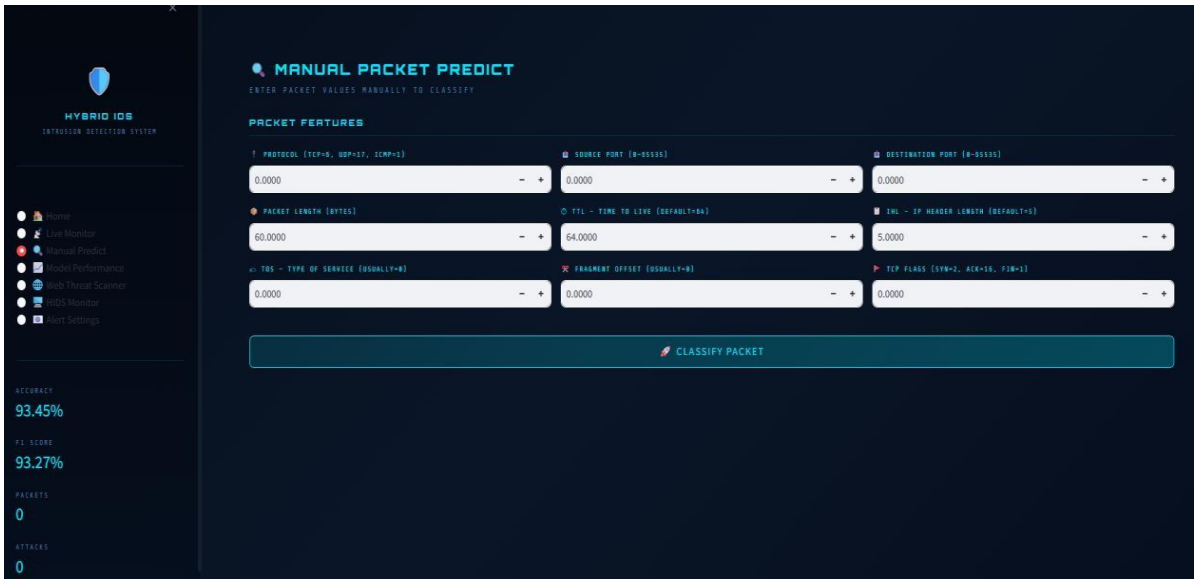

Screenshot of Home Dashboard - Figure 9.1

## 9.2 Live Network Monitor



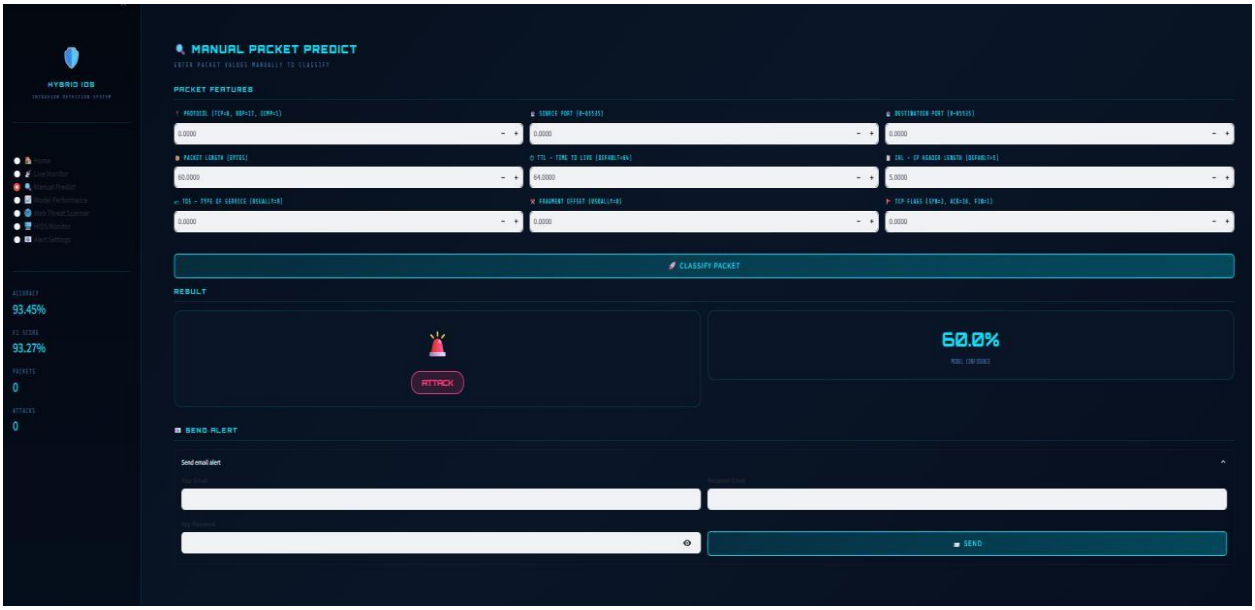

Screenshot of Live Monitor - Figure 9.2

## 9.3 Manual Packet Predict - Normal Result




Screenshot of Normal Prediction - Figure 9.3

## 9.4 Manual Packet Predict - Attack Result

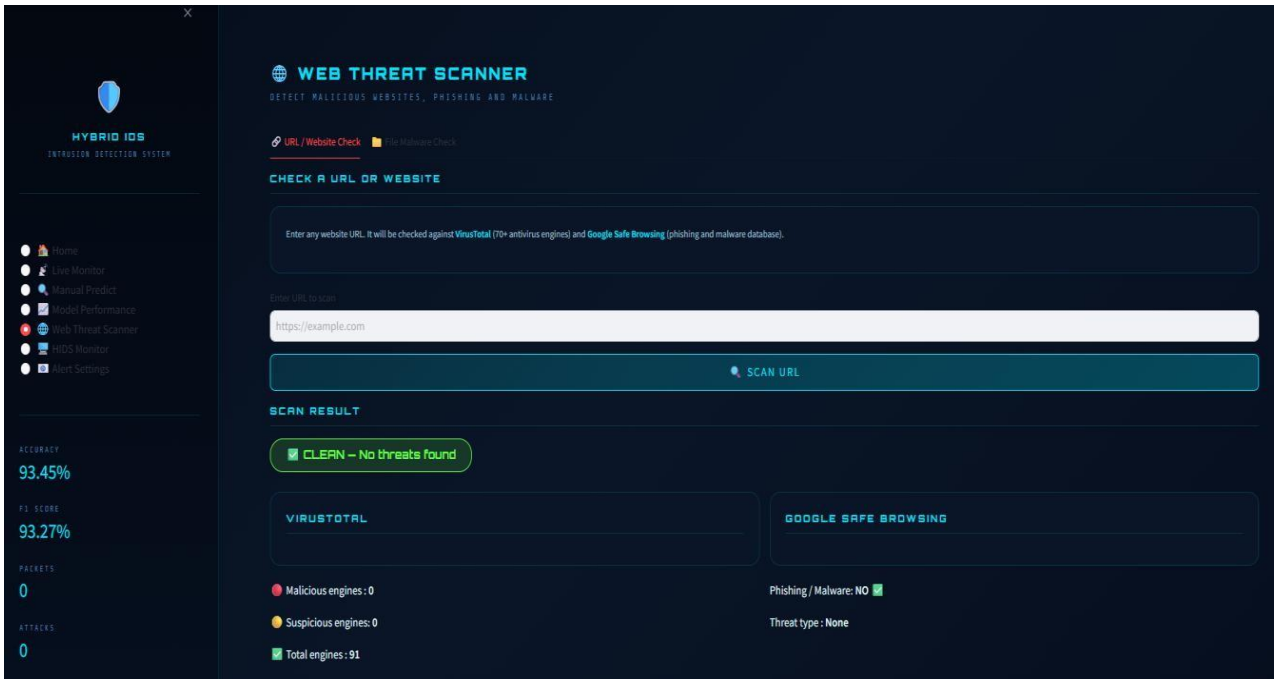



Screenshot of Attack Prediction - Figure 9.4

## 9.5 Model Performance Page

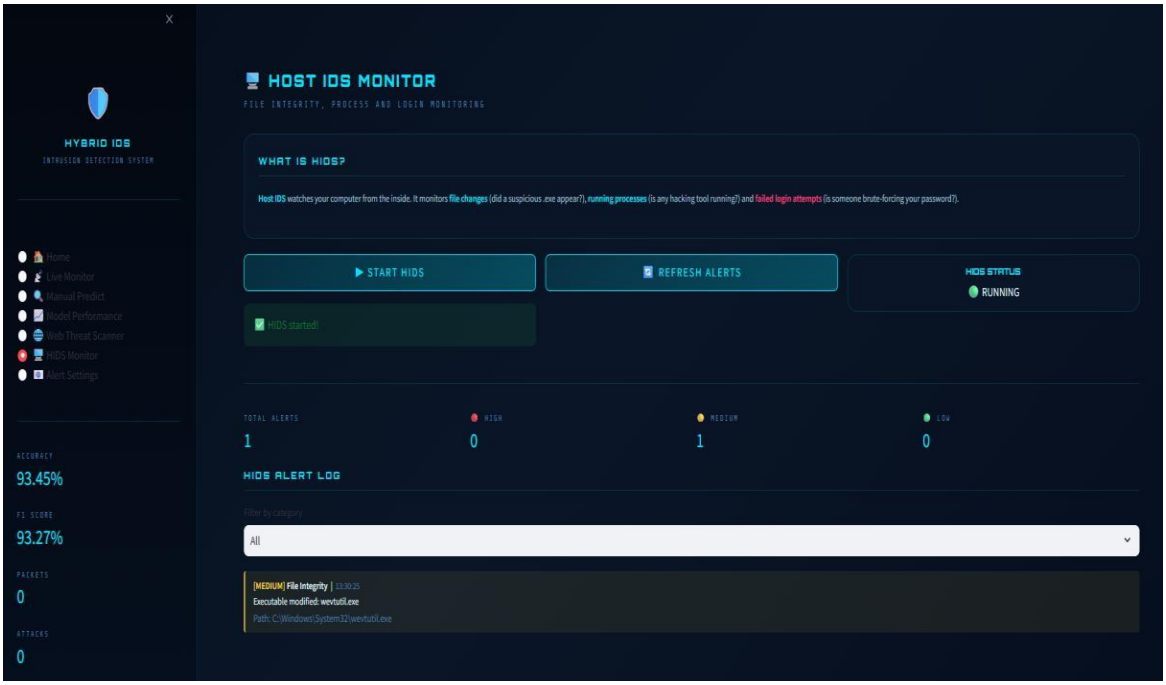

Screenshot of Model Performance - Figure 9.5

## 9.6 Web Threat Scanner



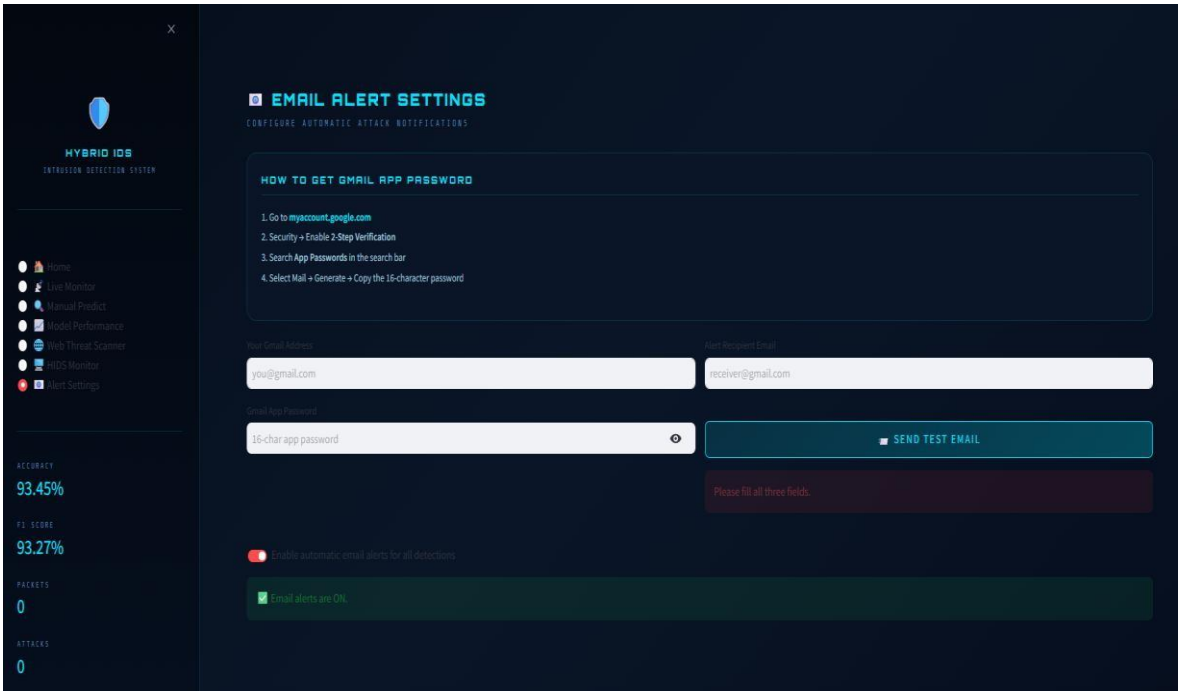

Screenshot of Web Threat Scanner - Figure 9.6

## 9.7 HIDS Monitor




Screenshot of HIDS Monitor - Figure 9.7

## 9.8 Email Alert Settings




Screenshot of Email Alert Settings - Figure 9.8